

- Description

This module manages a smart cold chain temperature monitoring system, ensuring that:

1. **The temperature inside a refrigerated storage unit is continuously monitored and maintained within a specified range.**
2. **If the temperature rises above the maximum threshold, an over-temperature warning is triggered.**
3. **If the temperature drops below the minimum threshold, a freezing alert is issued to prevent product damage.**
4. **A log of all temperature readings, alerts, and corrective actions is maintained for compliance and quality assurance.**

This system helps preserve perishable goods, reduce spoilage, and ensure compliance with food and pharmaceutical storage regulations.

- Specification

module ColdChainTemperatureMonitoringEx

var currentTemp, maxTemp, minTemp: real

var tempLog: sequence of string

process updateTemperature(changeAmount: real)

pre currentTemp + changeAmount $\geq$ -273.15

post

currentTemp = currentTemp + changeAmount

if (currentTemp > maxTemp) **then**

tempLog = tempLog $\oplus$ ("Over-temperature Warning: " + currentTemp + "°C, Risk of Spoilage.")

else if (currentTemp < minTemp) **then**

tempLog = tempLog $\oplus$ ("Freezing Alert: " + currentTemp + "°C, Possible Product Damage.")

else

tempLog = tempLog $\oplus$ ("Temperature Updated: " + currentTemp + "°C")

functional scenario form:

currentTemp <= maxTemp && Temperature updated successfully.

||

currentTemp > maxTemp && Over-temperature warning triggered, risk of spoilage.

||

currentTemp < minTemp && Freezing alert triggered, potential product damage.

end process

end module

- Java Code

```
package code;
```

```

import javax.swing.*;
import javax.swing.Timer;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.text.DecimalFormat;
import java.util.*;
import java.util.List;
import java.util.concurrent.ThreadLocalRandom;
/**
 * c121_ColdChainSystemEx
 *
 * Single-file, single-public-
class implementation for convenient compile/run.
 * All comments are in English (per request).
 *
 * Feature set (numbers-first design; condition-driven):
 * 1) Policy & limits: min/max temperature range, rate limits, door-
open grace,
 *     compressor ramp/short-
cycle protection, thermal inertia model, power outage handling.
 * 2) Sensors: ambient temperature, door open/close, evaporator efficie
ncy drift,
 *     supply power status (mains / brownout / outage), product thermal
mass factor.
 * 3) Control & protection: temperature update from sequences or sensor
fusion,
 *     anti-
overshoot clamping, hot/cold alarm bands, quick rise/fall, oscillation,
drift.
 * 4) Health & alerts: over-temp, under-temp (freezing risk), near-
boundary advisory,
 *     door-
open too long, compressor short cycling, brownout derate, outage warmin
g.
 * 5) Stats: rolling min/max/mean/variance plus p95; simple skewness/ku
rtosis approx.
 * 6) Logs: human-
readable lines and CSV export; bounded circular log to prevent OOM.
 * 7) Simulation: random ambient profile, door events, brownouts/outage
s, sensor noise.
 * 8) Optional Swing GUI: minimal dial/strip-
chart; toggle RUN_GUI to enable.

```

```

* 9) CLI harness: VT / FT / Z3 / UVT sequences switchable; defaults to
UVT on.
*
* CLI toggles at top of main():
*   RUN_VT/RUN_FT/RUN_Z3/RUN_UVT/RUN_RANDOM/RUN_GUI (boolean).
*
* CSV files:
*   coldchain_log_c121.csv, and GUI export coldchain_gui_export.csv (if
GUI used).
*/
public class c121_ColdChainSystemEx {
    // ===== Harness toggles =====
    private static final boolean RUN_VT      = false;
    private static final boolean RUN_FT      = false;
    private static final boolean RUN_Z3      = false;
    private static final boolean RUN_UVT     = true;
    private static final boolean RUN_RANDOM  = false; // random sensor-
driven sim
    private static final boolean RUN_GUI     = false; // requires deskto
p graphics
    private static final boolean VERBOSE_CONSOLE = true;
    public static void main(String[] args) {
        // 1) Policy (numbers-only rules)
        ColdPolicy policy = new ColdPolicy.Builder()
            .setMaxTemp(8.0)           // upper bound (°C)
            .setMinTemp(2.0)           // lower bound (°C)
            .setNearBand(0.6)          // near-
boundary advisory band (°C)
            .setMaxStepUp(12.0)        // max single-
step upward change (°C)
            .setMaxStepDown(-12.0)     // max single-
step downward change (°C)
            .setDoorOpenGraceSec(120)  // grace period before do
or alarm (s)
            .setShortCycleMinSec(180)  // minimum compressor OFF
→ON interval (s)
            .setBrownoutDerate(0.35)   // available cooling frac
tion under brownout
            .setWarmupPerOutageStep(0.6) // passive warming per st
ep during outage
            .setThermalInertia(0.55)   // inertia [0..1], higher
= slower to move
            .setOscWindow(8)           // oscillation detection w
indow
    }
}

```

```

        .setDriftWindow(16)           // drift trend window
        .setLeakSlopeThreshold(0.08) // rise trend threshold (
°C/step)

        .setLogCapacity(20000)       // bounded log size
        .build();

    // 2) Sensors (simple numeric models)
    ColdSensorSuite sensors = new ColdSensorSuite(
        new AmbientTempSensorC(24.0, 8.0), // sinusoidal
ambient
        new DoorSensorC(0.12, 25, 70),    // open prob, m
in/max open seconds
        new PowerSensorC(0.02, 0.02),    // brownout pro
b, outage prob
        new EvapEfficiencySensor(0.92, 0.06), // mean effici
ency, sigma
        new ProductMassSensor(1.0)       // relative the
rmal mass (1.0=nominal)
    );
    // 3) Controller
    ColdStorageEx storage = new ColdStorageEx(policy, 5.0 /* initia
l temp °C */);
    // 4) Demonstrative sensor-driven updates
    storage.updateWithSensors(sensors.sample()); // step 1
    storage.updateWithSensors(sensors.sample()); // step 2
    storage.updateWithSensors(sensors.sample()); // step 3
    // 5) Run harness
    // if (RUN_VT) ColdTestSuites.runVT(storage);
    // if (RUN_FT) ColdTestSuites.runFT(storage);
    // if (RUN_Z3) ColdTestSuites.runZ3(storage);
    // if (RUN_UVT) ColdTestSuites.runUVT(storage);
    //
    // if (RUN_RANDOM) {
    //     ColdRandomSimulator sim = new ColdRandomSimulator(sensors);
    //     sim.runSteps(storage, 240); // ~4 minutes with step=1s for
demonstration
    // }

    //VT
    ColdTestSuites.runVT(storage);

    //FT
    // ColdTestSuites.runFT(storage);

    //Z3

```

```

//      ColdTestSuites.runZ3(storage);

//UVT
//      ColdTestSuites.runUVT(storage);

// 6) Print Log (optional; can be Long)
storage.printTempLog();
// 7) Summary statistics
TempStats stats = storage.computeStats();
if (VERBOSE_CONSOLE) {
    System.out.println("\n=== Summary Statistics ===");
    System.out.printf(Locale.US, "count=%d, mean=%.2f°C, min=%.2f, max=%.2f, p95=%.2f%n",
        stats.count, stats.mean, stats.min, stats.max, stats.p95);
    System.out.printf(Locale.US, "deltaMean=%.2f, deltaMin=%.2f, deltaMax=%.2f, deltaP95=%.2f%n",
        stats.meanDelta, stats.minDelta, stats.maxDelta, stats.p95Delta);
    System.out.printf(Locale.US, "variance=%.3f, skew=%.3f, kurt=%.3f%n",
        stats.variance, stats.skewness, stats.kurtosis);
}
// 8) CSV export
try {
    String csvPath = "coldchain_log_c121.csv";
    storage.exportCsv(csvPath);
    if (VERBOSE_CONSOLE) System.out.println("CSV exported: " + csvPath);
} catch (IOException e) {
    System.out.println("CSV export failed: " + e.getMessage());
}
// 9) Optional GUI
if (RUN_GUI) {
    SwingUtilities.invokeLater(() -> {
        ColdFrame frame = new ColdFrame(storage, sensors);
        frame.setVisible(true);
    });
}
}

/* =====
*                               Data records
* =====

```

```

*/
class TempRecord {
    final long stepIndex;
    final double before;
    final double delta;
    final double after;
    final String message;
    final List<String> flags;
    final ColdSnapshot snapshot;
    final boolean compressorOn;
    TempRecord(long stepIndex, double before, double delta, double after,
        String message, List<String> flags, ColdSnapshot snapshot, boolean compressorOn) {
        this.stepIndex = stepIndex;
        this.before = before;
        this.delta = delta;
        this.after = after;
        this.message = message;
        this.flags = flags;
        this.snapshot = snapshot;
        this.compressorOn = compressorOn;
    }
}

class TempStats {
    final int count;
    final double mean, min, max, p95;
    final double meanDelta, minDelta, maxDelta, p95Delta;
    final double variance, skewness, kurtosis;
    TempStats(int count,
        double mean, double min, double max, double p95,
        double meanDelta, double minDelta, double maxDelta, double p95Delta,
        double variance, double skewness, double kurtosis) {
        this.count = count;
        this.mean = mean;
        this.min = min;
        this.max = max;
        this.p95 = p95;
        this.meanDelta = meanDelta;
        this.minDelta = minDelta;
        this.maxDelta = maxDelta;
        this.p95Delta = p95Delta;
        this.variance = variance;
    }
}

```

```

        this.skewness = skewness;
        this.kurtosis = kurtosis;
    }
    static TempStats from(List<TempRecord> list) {
        if (list.isEmpty()) {
            return new TempStats(0, Double.NaN, Double.NaN, Double.NaN,
                Double.NaN, Double.NaN, Double.NaN, Double.NaN,
                Double.NaN, Double.NaN, Double.NaN);
        }
        List<Double> vals = new ArrayList<>(list.size());
        List<Double> deltas = new ArrayList<>(list.size());
        double minV = Double.POSITIVE_INFINITY, maxV = Double.NEGATIVE_
INFINITY;
        double minD = Double.POSITIVE_INFINITY, maxD = Double.NEGATIVE_
INFINITY;
        double sumV = 0, sumD = 0;
        for (TempRecord r : list) {
            double v = r.after;
            double d = r.delta;
            vals.add(v);
            deltas.add(d);
            sumV += v; sumD += d;
            minV = Math.min(minV, v);
            maxV = Math.max(maxV, v);
            minD = Math.min(minD, d);
            maxD = Math.max(maxD, d);
        }
        double meanV = sumV / vals.size();
        double meanD = sumD / deltas.size();
        Collections.sort(vals);
        Collections.sort(deltas);
        double p95V = vals.get((int)Math.floor(0.95 * (vals.size()-
1)));
        double p95D = deltas.get((int)Math.floor(0.95 * (deltas.size()-
1)));
        // Simple moment-based approximations
        double m2=0, m3=0, m4=0;
        for (double v : vals) {
            double x = v - meanV;
            double x2 = x*x;
            m2 += x2;
            m3 += x2*x;
            m4 += x2*x2;

```

```

    }
    int n = vals.size();
    double variance = n > 1 ? m2/(n-1) : 0;
    double skew = (n>2 && variance>0) ? (Math.sqrt(n*(n-1))/(n-
2)) * (m3/Math.pow(m2,1.5)) : 0;
    double kurt = (n>3 && variance>0)
        ? (n*(n+1)*m4/(m2*m2*(n-1)*(n-2)*(n-3))) - 3.0*((n-
1)*(n-1))/((n-2)*(n-3)) : 0;
    return new TempStats(n, meanV, minV, maxV, p95V, meanD, minD, m
axD, p95D, variance, skew, kurt);
    }
}

/* =====
*
* Policy / limits
* =====
*/

class ColdPolicy {
    final double maxTemp;           // upper bound
    final double minTemp;           // lower bound
    final double nearBand;          // near-boundary advisory
    final double maxStepUp;          // per-step upper change clamp
    final double maxStepDown;        // per-step lower change clamp
    final int doorOpenGraceSec;      // grace time before door alarm
    final int shortCycleMinSec;      // min OFF→ON interval
    final double brownoutDerate;     // cooling fraction under brownout
    final double warmupPerOutageStep; // passive warming per step durin
g outage
    final double thermalInertia;     // fraction to filter deltas
    final int oscWindow;             // oscillation window
    final int driftWindow;           // drift/trend window
    final double leakSlopeThreshold; // slope threshold for upward drif
t
    final int logCapacity;
    private ColdPolicy(Builder b) {
        this.maxTemp = b.maxTemp;
        this.minTemp = b.minTemp;
        this.nearBand = b.nearBand;
        this.maxStepUp = b.maxStepUp;
        this.maxStepDown = b.maxStepDown;
        this.doorOpenGraceSec = b.doorOpenGraceSec;
        this.shortCycleMinSec = b.shortCycleMinSec;
        this.brownoutDerate = b.brownoutDerate;
        this.warmupPerOutageStep = b.warmupPerOutageStep;
        this.thermalInertia = b.thermalInertia;
    }
}

```



```

        this.oscWindow = b.oscWindow;
        this.driftWindow = b.driftWindow;
        this.leakSlopeThreshold = b.leakSlopeThreshold;
        this.logCapacity = b.logCapacity;
    }
    boolean nearUpper(double t) { return t > (maxTemp - nearBand); }
    boolean nearLower(double t) { return t < (minTemp + nearBand); }
    static class Builder {
        private double maxTemp=8.0, minTemp=2.0, nearBand=0.5;
        private double maxStepUp=12.0, maxStepDown=-12.0;
        private int doorOpenGraceSec=120, shortCycleMinSec=180;
        private double brownoutDerate=0.35, warmupPerOutageStep=0.5, thermalInertia=0.5;
        private int oscWindow=8, driftWindow=16;
        private double leakSlopeThreshold=0.08;
        private int logCapacity=20000;
        Builder setMaxTemp(double v){this.maxTemp=v;return this;}
        Builder setMinTemp(double v){this.minTemp=v;return this;}
        Builder setNearBand(double v){this.nearBand=v;return this;}
        Builder setMaxStepUp(double v){this.maxStepUp=v;return this;}
        Builder setMaxStepDown(double v){this.maxStepDown=v;return this;}
    };
        Builder setDoorOpenGraceSec(int v){this.doorOpenGraceSec=v;return this;}
        Builder setShortCycleMinSec(int v){this.shortCycleMinSec=v;return this;}
        Builder setBrownoutDerate(double v){this.brownoutDerate=v;return this;}
        Builder setWarmupPerOutageStep(double v){this.warmupPerOutageStep=v;return this;}
        Builder setThermalInertia(double v){this.thermalInertia=v;return this;}
        Builder setOscWindow(int v){this.oscWindow=v;return this;}
        Builder setDriftWindow(int v){this.driftWindow=v;return this;}
        Builder setLeakSlopeThreshold(double v){this.leakSlopeThreshold=v;return this;}
        Builder setLogCapacity(int v){this.logCapacity=v;return this;}
        ColdPolicy build(){return new ColdPolicy(this);}
    }
}

/* =====
 *
 * Sensors & snapshot
 * =====
 */

```

```

class ColdSnapshot {
    final double ambientTemp;    // °C
    final boolean doorOpen;      // door status
    final int doorOpenSec;       // consecutive open seconds (approx)
    final PowerStatus power;     // mains/brownout/outage
    final double evapEff;        // [0..1] effective cooling factor
    final double productMass;    // multiplier for inertia (>=0.5..2.0)
    final double suggestedDelta; // suggested per-
    step  $\Delta T$  from sensor fusion
    ColdSnapshot(double ambientTemp, boolean doorOpen, int doorOpenSec,
        PowerStatus power,
            double evapEff, double productMass, double suggestedDe
lta) {
        this.ambientTemp = ambientTemp;
        this.doorOpen = doorOpen;
        this.doorOpenSec = doorOpenSec;
        this.power = power;
        this.evapEff = evapEff;
        this.productMass = productMass;
        this.suggestedDelta = suggestedDelta;
    }
}

enum PowerStatus { NORMAL, BROWNOUT, OUTAGE }

class AmbientTempSensorC {
    private final double base;
    private final double amplitude;
    private int t=0;
    AmbientTempSensorC(double base, double amplitude){this.base=base;th
is.amplitude=amplitude;}
    double read() {
        t++;
        double phase = Math.sin((t % 1440) * Math.PI/720.0);
        return base + amplitude * phase + ThreadLocalRandom.current().n
extGaussian()*0.4;
    }
}

class DoorSensorC {
    private final double openProb; // probability per step to toggle op
    en burst
    private final int minOpenSec, maxOpenSec;
    private int remaining = 0;
    private final Random rnd = new Random(1211);
    DoorSensorC(double openProb, int minOpenSec, int maxOpenSec){

```

```

        this.openProb=openProb;this.minOpenSec=minOpenSec;this.maxOpenSec=maxOpenSec;
    }
    boolean isOpen() {
        if (remaining>0){remaining--; return true;}
        if (rnd.nextDouble()<openProb){
            remaining = minOpenSec + rnd.nextInt(Math.max(1,maxOpenSec-minOpenSec+1));
            return true;
        }
        return false;
    }
    int openSeconds() { return remaining>0? remaining : 0; }
}

class PowerSensorC {
    private final double brownoutProb, outageProb;
    private final Random rnd = new Random(1212);
    PowerSensorC(double brownoutProb, double outageProb){
        this.brownoutProb=brownoutProb;this.outageProb=outageProb;
    }
    PowerStatus read() {
        double r = rnd.nextDouble();
        if (r < outageProb) return PowerStatus.OUTAGE;
        if (r < outageProb + brownoutProb) return PowerStatus.BROWNOUT;
        return PowerStatus.NORMAL;
    }
}

class EvapEfficiencySensor {
    private final double mean, sigma;
    private final Random rnd = new Random(1213);
    EvapEfficiencySensor(double mean, double sigma){this.mean=mean;this.sigma=sigma;}
    double read(){
        double v = rnd.nextGaussian()*sigma + mean;
        if (v < 0.6) v = 0.6 + Math.abs(v-0.6)*0.2;
        if (v > 1.05) v = 1.05;
        return v;
    }
}

class ProductMassSensor {
    private final double base;
    ProductMassSensor(double base){this.base = Math.max(0.5, Math.min(2.0, base));}
}

```

```

    double read(){ return base; } // could be random within bounds if desired
}
/** Aggregation of sensors, plus simple fusion to suggest  $\Delta T$ . */
class ColdSensorSuite {
    private final AmbientTempSensorC ambient;
    private final DoorSensorC door;
    private final PowerSensorC power;
    private final EvapEfficiencySensor evap;
    private final ProductMassSensor product;
    ColdSensorSuite(AmbientTempSensorC ambient, DoorSensorC door, PowerSensorC power,
                    EvapEfficiencySensor evap, ProductMassSensor product) {
        this.ambient = ambient; this.door = door; this.power = power; this.evap = evap; this.product = product;
    }
    ColdSnapshot sample() {
        double amb = ambient.read();
        boolean isOpen = door.isOpen();
        int openSec = door.openSeconds();
        PowerStatus ps = power.read();
        double eff = evap.read();
        double mass = product.read();
        // Very simple fusion model (numbers-first):
        // - base cooling tendency toward 4°C if power is normal
        // - reduced cooling under brownout; no active cooling under outage
        // - door open increases warming tendency proportional to ambient delta
        // - evaporator efficiency scales cooling magnitude
        // - product mass increases inertia (handled later)
        double toward = 4.0; // nominal setpoint (informational)
        double cooling = -0.45 * eff; // negative delta = cooling
        if (ps == PowerStatus.BROWNOUT) cooling *= 0.35;
        if (ps == PowerStatus.OUTAGE) cooling = 0.0;
        double warmingFromAmbient = Math.max(0.0, (amb - toward)) * 0.015; // slow ingress
        if (isOpen) warmingFromAmbient *= 2.2; // door amplification
        double suggested = cooling + warmingFromAmbient;
        return new ColdSnapshot(amb, isOpen, openSec, ps, eff, mass, suggested);
    }
}

```

```

/* =====
 *
 *                      Anomaly detection
 * =====
 */
class ColdAnomalyDetector {
    static List<String> detect(ColdPolicy p, double before, double delta,
        double after,
                                List<TempRecord> hist, ColdSnapshot s,
                                boolean compressorOn, int secondsSinceCompressorOn,
                                int secondsSinceCompressorOff) {
        List<String> flags = new ArrayList<>();
        // Step limits
        if (delta > p.maxStepUp) flags.add("Rapid rise: step delta exceeds maxStepUp");
        if (delta < p.maxStepDown) flags.add("Rapid fall: step delta below maxStepDown");
        // Basic bounds
        if (after > p.maxTemp) {
            flags.add("WARNING: Over-temperature detected! Risk of spoilage.");
        } else if (after < p.minTemp) {
            flags.add("ALERT: Freezing risk! Possible product damage.");
        };
        if (p.nearUpper(after)) flags.add("Advisory: near upper bound");
        if (p.nearLower(after)) flags.add("Advisory: near lower bound");
        }
        // Door-open handling
        if (s != null && s.doorOpen) {
            if (s.doorOpenSec > p.doorOpenGraceSec) flags.add("Door open too long");
            else flags.add("Door open (within grace)");
        }
        // Power states
        if (s != null) {
            if (s.power == PowerStatus.BROWNOUT) flags.add("Brownout derate active");
            if (s.power == PowerStatus.OUTAGE) flags.add("Power outage: passive warming");
        }
        // Compressor short cycle protection

```

```

        if (compressorOn && secondsSinceCompressorOn < 3) {
            flags.add("Compressor just engaged");
        }
        if (!compressorOn && secondsSinceCompressorOff < p.shortCycleMinSec) {
            flags.add("Short-cycle prevention: delaying restart");
        }
        // Oscillation: sign flip pattern with sufficient amplitude
        if (hist.size() >= p.oscWindow) {
            if (isOscillating(hist, p.oscWindow)) flags.add("Oscillation pattern detected");
        }
        // Drift / Leak tendency
        if (hist.size() >= p.driftWindow) {
            double slope = estimateSlope(hist, p.driftWindow);
            if (slope > p.leakSlopeThreshold) flags.add("Upward drift detected");
            if (slope < -p.leakSlopeThreshold) flags.add("Downward drift detected");
        }
        return flags;
    }

    private static boolean isOscillating(List<TempRecord> hist, int window) {
        int n = hist.size();
        int flips = 0;
        for (int i = n - window + 1; i < n; i++) {
            double d1 = hist.get(i-1).delta;
            double d2 = hist.get(i).delta;
            if (Math.signum(d1) != Math.signum(d2) && Math.abs(d1) > 2.0 && Math.abs(d2) > 2.0) {
                flips++;
            }
        }
        return flips >= (window/2);
    }

    private static double estimateSlope(List<TempRecord> hist, int k) {
        int n = Math.min(k, hist.size());
        double sumX=0, sumY=0, sumXY=0, sumX2=0;
        for (int i=0;i<n;i++){
            double x=i;
            double y=hist.get(hist.size()-1-i).after;
            sumX+=x; sumY+=y; sumXY+=x*y; sumX2+=x*x;
        }
    }

```

```

        double denom = n*sumX2 - sumX*sumX;
        if (denom==0) return 0;
        return (n*sumXY - sumX*sumY)/denom;
    }
}

/* =====
 *
 *                      Core ColdStorageEx
 * =====
 */
class ColdStorageEx {
    private final ColdPolicy policy;
    private double currentTemp;
    private final CircularBuffer121<TempRecord> log;
    private long step = 0;
    // Simplified compressor state machine (numbers-first)
    private boolean compressorOn = false;
    private int secondsSinceOn = 9999;
    private int secondsSinceOff = 9999;
    ColdStorageEx(ColdPolicy policy, double initialTemp) {
        this.policy = policy;
        this.currentTemp = initialTemp;
        this.log = new CircularBuffer121<>(policy.logCapacity);
    }
    /** Original API semantics preserved: update by a delta amount. */
    public void updateTemperature(double changeAmount) {
        updateInternal(changeAmount, null);
    }
    /** Sensor-
driven update: fuse sensors to form a suggested delta, then apply polic
y. */
    public void updateWithSensors(ColdSnapshot s) {
        // Thermal inertia blending (numbers-only, no PID)
        double rawDelta = s.suggestedDelta;
        // Power handling: outage warms passively
        if (s.power == PowerStatus.OUTAGE) {
            rawDelta += policy.warmupPerOutageStep;
        }
        // Door open increases exchange rate; emulate by attenuation of
cooling
        if (s.doorOpen) {
            if (rawDelta < 0) rawDelta *= 0.8;
            else rawDelta *= 1.3;
        }
        // Product mass → more inertia (reduce magnitude of delta)

```

```

        rawDelta *= (1.0 - 0.35 * (s.productMass - 1.0)); // mass>1 =>
reduce |delta|
        // Compressor gate: simple setpoint-like behavior around 4.0°C
        double setpoint = 4.0;
        if (currentTemp > setpoint + 0.3 && secondsSinceOff >= policy.s
hortCycleMinSec) {
            compressorOn = true; secondsSinceOn = 0;
        } else if (currentTemp < setpoint - 0.3) {
            compressorOn = false; secondsSinceOff = 0;
        }
        // If compressor is off, reduce cooling component
        if (!compressorOn && rawDelta < 0) rawDelta *= 0.4;
        // Apply thermal inertia blend: after = before + (1 - inertia)*
rawDelta
        double inertia = clamp(policy.thermalInertia * s.productMass, 0
.2, 0.9);
        double blended = rawDelta * (1.0 - inertia);
        updateInternal(blended, s);
        // Advance compressor timers
        secondsSinceOn++;
        secondsSinceOff++;
    }
    private static double clamp(double v, double lo, double hi){
        return Math.max(lo, Math.min(hi, v));
    }
    private void updateInternal(double changeAmount, ColdSnapshot snaps
hot) {
        double before = currentTemp;
        // Clamp single-step delta
        double delta = Math.max(policy.maxStepDown, Math.min(policy.max
StepUp, changeAmount));
        double after = before + delta;
        // Build base message
        String base = String.format(Locale.US,
            "Step #%d | Before: %.2f°C | Δ: %+,.2f°C | Current Temp
erature: %.2f°C",
            ++step, before, delta, after);
        // Build flags
        List<TempRecord> history = log.snapshotView();
        List<String> flags = ColdAnomalyDetector.detect(
            policy, before, delta, after, history, snapshot,
            compressorOn, secondsSinceOn, secondsSinceOff
        );
    }

```



```

        // Hard bound clamping to prevent uncontrolled drifts beyond ph
ysical window
        if (after > policy.maxTemp + 30.0) after = policy.maxTemp + 30.
0; // wide safety clamp
        if (after < policy.minTemp - 30.0) after = policy.minTemp - 30.
0;

        // Compose message
        StringBuilder sb = new StringBuilder(base);
        if (!flags.isEmpty()) {
            sb.append(" | ");
            for (int i=0;i<flags.size();i++){
                if (i>0) sb.append("; ");
                sb.append(flags.get(i));
            }
        }
        String line = sb.toString();
        System.out.println(line);
        currentTemp = after;
        // Compose snapshot if null (for CSV column integrity)
        ColdSnapshot snap = (snapshot != null) ? snapshot :
            new ColdSnapshot(Double.NaN, false, 0, PowerStatus.NORM
AL, Double.NaN, 1.0, Double.NaN);
        TempRecord rec = new TempRecord(step, before, delta, after, lin
e, flags, snap, compressorOn);
        log.add(rec);
    }
    public void printTempLog() {
        System.out.println("\nCold Storage Temperature Log:");
        for (TempRecord r : log.iterable()) {
            System.out.println(r.message);
        }
    }
    public TempStats computeStats() {
        return TempStats.from(log.asList());
    }
    public void exportCsv(String file) throws IOException {
        try (PrintWriter pw = new PrintWriter(new BufferedWriter(new Fi
leWriter(file)))) {
            pw.println("step,before,delta,after,has_flags,flags,ambient
,doorOpen,doorOpenSec,power,evapEff,productMass,suggestedDelta,compress
orOn");
            for (TempRecord r : log.iterable()) {
                String flags = String.join("|", r.flags);

```

```

        pw.printf(Locale.US, "%d,%.6f,%.6f,%.6f,%b,%s,%.6f,%b,%d,%s,%.6f,%.6f,%.6f,%b%n",
            r.stepIndex, r.before, r.delta, r.after, !r.flags.isEmpty(),
            escapeCsv(flags),
            r.snapshot.ambientTemp, r.snapshot.doorOpen, r.snapshot.doorOpenSec,
            r.snapshot.power.name(),
            r.snapshot.evapEff, r.snapshot.productMass, r.snapshot.suggestedDelta,
            r.compressorOn);
    }
}

private static String escapeCsv(String s) {
    if (s==null) return "";
    if (s.contains(",") || s.contains("\"")) {
        return "\"" + s.replace("\"", "\\\"") + "\"";
    }
    return s;
}

}

/* =====
 *                      Circular buffer
 * =====
 */

class CircularBuffer121<T> {
    private final Object[] arr;
    private int head = 0; // next write
    private int size = 0;
    CircularBuffer121(int capacity) {
        this.arr = new Object[Math.max(256, capacity)];
    }
    void add(T v) {
        arr[head] = v;
        head = (head + 1) % arr.length;
        if (size < arr.length) size++;
    }
    @SuppressWarnings("unchecked")
    List<T> asList() {
        List<T> list = new ArrayList<>(size);
        for (int i = size - 1; i >= 0; i--) {
            int idx = (head - 1 - i + arr.length) % arr.length;
            list.add((T) arr[idx]);
        }
    }
}

```

```

        }
        return list;
    }
    Iterable<T> iterable() {
        return this::iterator;
    }
    private Iterator<T> iterator() {
        List<T> snap = asList();
        return snap.iterator();
    }
    List<T> snapshotView() {
        return asList();
    }
}

/* =====
 *                      Random simulator
 * =====
 */
class ColdRandomSimulator {
    private final ColdSensorSuite sensors;
    ColdRandomSimulator(ColdSensorSuite s){this.sensors=s;}
    void runSteps(ColdStorageEx t, int steps){
        for (int i=0;i<steps;i++){
            t.updateWithSensors(sensors.sample());
        }
    }
}

/* =====
 *                      Swing GUI (optional)
 * =====
 */
class ColdFrame extends JFrame {
    private final ColdPanel panel;
    private final ColdStorageEx storage;
    private final ColdSensorSuite sensors;
    private volatile boolean running = true;
    private final Timer timer;
    ColdFrame(ColdStorageEx storage, ColdSensorSuite sensors) {
        super("Cold Chain Monitor (c121)");
        this.storage = storage;
        this.sensors = sensors;
        this.panel = new ColdPanel();
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setSize(860, 560);
    }

```

```

        setLocationRelativeTo(null);
        setLayout(new BorderLayout());
        add(panel, BorderLayout.CENTER);
        JPanel controls = new JPanel(new FlowLayout(FlowLayout.LEFT));
        controls.setBorder(new EmptyBorder(8,8,8,8));
        JButton stepBtn = new JButton("Step");
        JButton runBtn = new JButton("Run");
        JButton stopBtn = new JButton("Stop");
        JButton exportBtn = new JButton("Export CSV");
        controls.add(stepBtn);
        controls.add(runBtn);
        controls.add(stopBtn);
        controls.add(exportBtn);
        add(controls, BorderLayout.SOUTH);
        stepBtn.addActionListener(e -> doStep());
        runBtn.addActionListener(e -> running = true);
        stopBtn.addActionListener(e -> running = false);
        exportBtn.addActionListener(e -> {
            try {
                storage.exportCsv("coldchain_gui_export.csv");
                JOptionPane.showMessageDialog(this, "Exported to coldchain_gui_export.csv");
            } catch (IOException ex) {
                JOptionPane.showMessageDialog(this, "Export failed: " + ex.getMessage());
            }
        });
        timer = new Timer(150, e -> { if (running) doStep(); });
        addWindowListener(new WindowAdapter() {
            @Override public void windowOpened(WindowEvent e) { timer.start(); }
            @Override public void windowClosed(WindowEvent e) { timer.stop(); }
        });
    }
    private void doStep() {
        storage.updateWithSensors(sensors.sample());
        panel.feed(storage);
        panel.repaint();
    }
}

class ColdPanel extends JPanel {
    private final Deque<Double> series = new ArrayDeque<>();
    private static final int CAP = 180;

```

```

private double last = 5.0;
private final DecimalFormat df = new DecimalFormat("#0.0");
void feed(ColdStorageEx storage) {
    // This is a lightweight panel; we do not access internals. We
    emulate last temp by
    // stretching the drawn series rather than reading controller s
    tate directly.
    // For a real UI, you'd expose a getter or subscribe to events.
    // Here we visually follow the trend using a small random walk
    around last.
    double next = last + ThreadLocalRandom.current().nextGaussian()
*0.25;
    last = next;
    if (series.size()>=CAP) series.pollFirst();
    series.addLast(next);
}
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    // background
    g.setColor(new Color(245,247,250));
    g.fillRect(0,0,getWidth(),getHeight());
    // grid
    g.setColor(new Color(220,225,232));
    for (int x=40;x<getWidth();x+=40) g.drawLine(x,20,x,getHeight()
-40);
    for (int y=20;y<getHeight()-
40;y+=40) g.drawLine(40,y,getWidth()-20,y);
    int w = getWidth()-60, h = getHeight()-80;
    int ox=40, oy=getHeight()-40;
    // derive min/max
    double min = series.isEmpty()? 2.0 : Collections.min(series);
    double max = series.isEmpty()? 8.0 : Collections.max(series);
    double span = Math.max(1e-6, max-min);
    // curve
    g.setColor(new Color(72,120,232));
    int i=0, px=-1, py=-1;
    for (double v : series) {
        int x = ox + (int)((i*1.0/(CAP-1))*w);
        int y = oy - (int)((v - min)/span)*h);
        if (px>=0) g.drawLine(px,py,x,y);
        px=x;py=y;i++;
    }
    // labels

```

```

        g.setColor(new Color(50,50,60));
        g.drawString("Temperature (°C, scaled)", 10, 16);
        g.drawString("min="+df.format(min)+" max="+df.format(max), 10,
getHeight()-10);
    }
}
/* =====
 *
 *                      Test suites (reuse your sequences)
 * =====
 */
class ColdTestSuites {
    static void runVT(ColdStorageEx s) { for (double d: VT_SEQ) s.updateTemperature(d); }
    static void runZ3(ColdStorageEx s) { for (double d: Z3_SEQ) s.updateTemperature(d); }
    static void runUVT(ColdStorageEx s){ for (double d: UVT_SEQ) s.updateTemperature(d); }
    static void runFT(ColdStorageEx s) { for (double d: FT_SEQ) s.updateTemperature(d); }
    // ==== Provided VT sequence (commented in original; kept here) ====
    =
    static final double[] VT_SEQ = new double[]{
        -26.8,-23.9,-11.3,19.6,-18.9,7.8,33.3,-21.6,35.9,-12.0,22.8,-
36.4,-36.8,33.4,31.6,-37.8,-36.3,-39.7,-47.4,-4.0,
        -31.3,-46.6,45.7,-28.0,-42.6,27.9,10.3,21.4,-
0.1,47.8,21.4,37.4,-45.7,16.6,-18.3,-33.2,2.3,-2.3,46.1,-13.9,
        36.7,-33.3,-27.5,28.3,16.7,33.2,-46.2,1.8,27.0,-37.8,-2.8,-
23.8,-0.0,22.2,-25.0,-23.6,-8.7,-14.7,49.3,-48.8,
        -6.3,-17.7,-7.3,18.1,-28.2,-30.0,-39.6,-
0.3,34.7,42.2,2.5,46.8,11.0,25.3,28.9,26.9,-21.0,19.1,20.3,33.3,-40.5,
        -13.4,-47.7,46.1,-20.5,17.8,16.5,-23.2,19.3,-24.2,-
50.0,49.5,20.1,40.7,-33.3,-38.7,31.8,-47.4,1.3,-37.5
    };
    // ==== Provided Z3 subset (same as VT here per original structure)
    ====
    static final double[] Z3_SEQ = new double[]{
        -26.8,-23.9,-11.3,19.6,-18.9,7.8,33.3,-21.6,35.9,-12.0,22.8,-
36.4,-36.8,33.4,31.6,-37.8,-36.3,-39.7,-47.4,-4.0,
        -31.3,-46.6,45.7,-28.0,-42.6,27.9,10.3,21.4,-
0.1,47.8,21.4,37.4,-45.7,16.6,-18.3,-33.2,2.3,-2.3,46.1,-13.9,
        36.7,-33.3,-27.5,28.3,16.7,33.2,-46.2,1.8,27.0,-37.8,-2.8,-
23.8,-0.0,22.2,-25.0,-23.6,-8.7,-14.7,49.3,-48.8,
        -6.3,-17.7,-7.3,18.1,-28.2,-30.0,-39.6,-
0.3,34.7,42.2,2.5,46.8,11.0,25.3,28.9,26.9,-21.0,19.1,20.3,33.3,-40.5,

```

[illegible]

[illegible][illegible][illegible][illegible][illegible]

[illegible]

—

[illegible]

—

[illegible]

—

[illegible]

—

[illegible]

—

[illegible]

—

[illegible]

—

[illegible]

[illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible]

[illegible][illegible][illegible][illegible][illegible][illegible]

0.00,

0.00,

[illegible][illegible][illegible][illegible][illegible][illegible][illegible]

[illegible]

0.00,

[illegible][illegible][illegible][illegible][illegible][illegible]

[illegible][illegible][illegible][illegible][illegible][illegible][illegible]

[illegible][illegible][illegible][illegible][illegible]

—

[illegible]

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c121_ColdChainSystemEx.java	47.1 %	80	90	170
> ColdTestSuites	25.0 %	2	6	8
> ColdStorageEx	40.0 %	16	24	40
> ColdPolicy	50.0 %	2	2	4
> EvapEfficiencySensor	50.0 %	2	2	4
> PowerSensorC	50.0 %	2	2	4
> ColdSensorSuite	66.7 %	4	2	6
> CircularBuffer121<T>	75.0 %	3	1	4
> ColdAnomalyDetector	76.8 %	43	13	56
> DoorSensorC	100.0 %	6	0	6
> AmbientTempSensorC		0	0	0
> c121_ColdChainSystemEx		0	0	0
> ColdSnapshot		0	0	0
> PowerStatus		0	0	0
> ProductMassSensor		0	0	0

5. Fuzz testing instruction coverage: 55.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c121_ColdChainSystemEx.java	68.2 %	2,559	1,195	3,754
> ColdStorageEx	55.5 %	286	229	515
> EvapEfficiencySensor	76.1 %	35	11	46
> ColdAnomalyDetector	85.9 %	256	42	298
> PowerSensorC	89.5 %	34	4	38
> CircularBuffer121<T>	90.5 %	86	9	95
> ColdSensorSuite	93.3 %	83	6	89
> ColdPolicy	95.2 %	79	4	83
> ColdTestSuites	95.3 %	1,289	63	1,352
> c121_ColdChainSystemEx	96.5 %	83	3	86
> AmbientTempSensorC	100.0 %	42	0	42
> ColdSnapshot	100.0 %	24	0	24
> DoorSensorC	100.0 %	68	0	68
> PowerStatus	100.0 %	34	0	34
> ProductMassSensor	100.0 %	13	0	13

6. Z3 branch coverage: 62.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c121_ColdChainSystemEx.java	58.2 %	99	71	170
> ColdTestSuites	25.0 %	2	6	8
> EvapEfficiencySensor	50.0 %	2	2	4
> PowerSensorC	50.0 %	2	2	4
> ColdStorageEx	62.5 %	25	15	40
> TempStats	62.5 %	10	6	16
> ColdSensorSuite	66.7 %	4	2	6
> CircularBuffer121<T>	75.0 %	3	1	4
> ColdAnomalyDetector	75.0 %	42	14	56
> ColdPolicy	75.0 %	3	1	4
> DoorSensorC	100.0 %	6	0	6
> AmbientTempSensorC		0	0	0
> c121_ColdChainSystemEx		0	0	0
> ColdSnapshot		0	0	0
> PowerStatus		0	0	0

7. Z3 instruction coverage: 90.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c121_ColdChainSystemEx.java	83.4 %	3,484	695	4,179
> EvapEfficiencySensor	76.1 %	35	11	46
> ColdAnomalyDetector	85.2 %	254	44	298
> PowerSensorC	89.5 %	34	4	38
> ColdStorageEx	90.7 %	467	48	515
> ColdSensorSuite	93.3 %	83	6	89
> TempStats	94.3 %	316	19	335
> c121_ColdChainSystemEx	95.6 %	194	9	203
> ColdTestSuites	96.2 %	1,597	63	1,660
> ColdPolicy	97.6 %	81	2	83
> AmbientTempSensorC	100.0 %	42	0	42
> CircularBuffer121<T>	100.0 %	95	0	95
> ColdSnapshot	100.0 %	24	0	24
> DoorSensorC	100.0 %	68	0	68
> PowerStatus	100.0 %	34	0	34

8. Unimproved V-method branch coverage: 45%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c121_ColdChainSystemEx.java	48.2 %	82	88	170
> ColdTestSuites	25.0 %	2	6	8
> ColdStorageEx	45.0 %	18	22	40
> ColdPolicy	50.0 %	2	2	4
> EvapEfficiencySensor	50.0 %	2	2	4
> PowerSensorC	50.0 %	2	2	4
> ColdSensorSuite	66.7 %	4	2	6
> CircularBuffer121<T>	75.0 %	3	1	4
> ColdAnomalyDetector	76.8 %	43	13	56
> DoorSensorC	100.0 %	6	0	6
> AmbientTempSensorC		0	0	0
> c121_ColdChainSystemEx		0	0	0
> ColdSnapshot		0	0	0
> PowerStatus		0	0	0
> ProductMassSensor		0	0	0

9. Unimproved V-method instruction coverage: 59.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c121_ColdChainSystemEx.java	69.0 %	2,591	1,165	3,756
> ColdStorageEx	59.6 %	307	208	515
> EvapEfficiencySensor	76.1 %	35	11	46
> ColdAnomalyDetector	85.9 %	256	42	298
> PowerSensorC	89.5 %	34	4	38
> ColdSensorSuite	93.3 %	83	6	89
> ColdPolicy	95.2 %	79	4	83
> ColdTestSuites	95.3 %	1,289	63	1,352
> c121_ColdChainSystemEx	96.6 %	85	3	88
> AmbientTempSensorC	100.0 %	42	0	42
> CircularBuffer121<T>	100.0 %	95	0	95
> ColdSnapshot	100.0 %	24	0	24
> DoorSensorC	100.0 %	68	0	68
> PowerStatus	100.0 %	34	0	34
> ProductMassSensor	100.0 %	13	0	13